

Halving the World

How a coordinate becomes a string of bits: halving, binary fractions, normalization, and the multiply-and-truncate trick

One way to store a coordinate in a fixed number of bits is to describe it as a sequence of choices: which half of the range it is in, then which half of that half, and so on. This note explains that idea and shows why it doesn't need a loop. The answers to the halving questions are the binary digits of the coordinate's position within its range, so they can be read off with one subtraction, one division, one multiplication and one truncation.

1. Halving the range

Take a longitude, which lies between -180° and 180° . Ask whether it is in the upper half of the range. If it is, write down 1 and keep the upper half. Otherwise write 0 and keep the lower half. Then ask the same question about the half you kept.

Each answer is one bit, and each answer halves the interval. After n answers the range has effectively been cut into 2^n equal cells, and the n bits name exactly one of them. Figure 1 does this for longitude 100° with six questions.

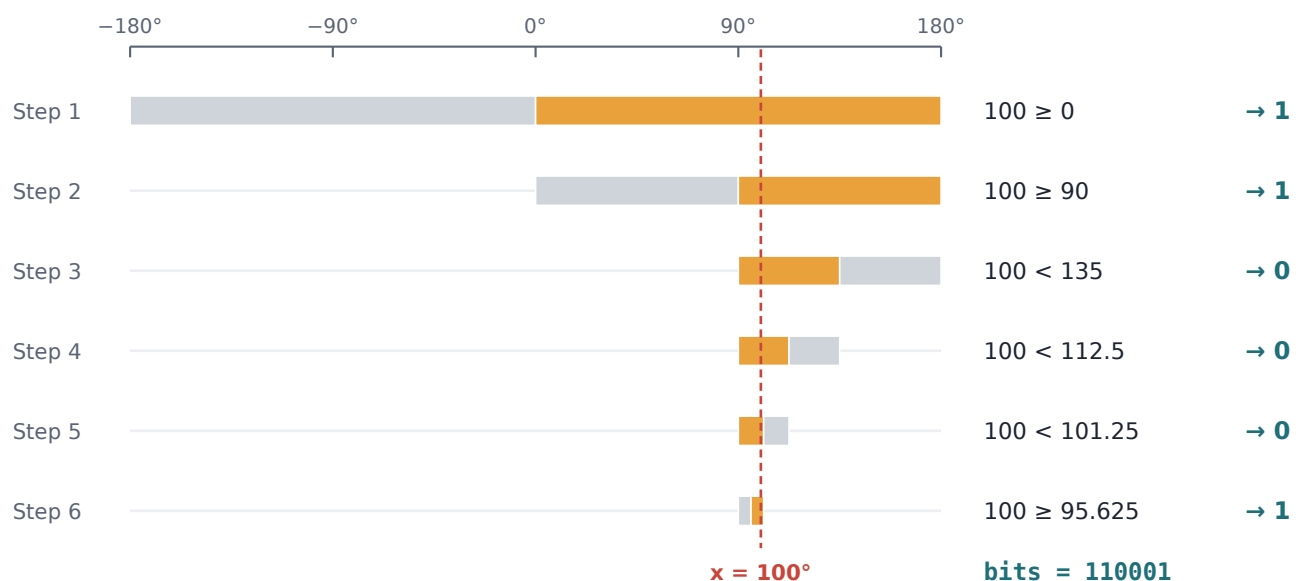


Figure 1. Six halvings locate longitude 100° (red) in the cell $[95.625^\circ, 101.25^\circ]$. Amber marks the half kept at each step, drawn to scale.

Implemented literally, this is a loop that computes a midpoint, compares and narrows the interval n times. The rest of this note shows that the loop is unnecessary, because the answers it produces already exist as the binary digits of a single number.

2. Digits are “which slice?” answers

Decimal notation works the same way, with ten slices instead of two. Take 0.372 and split $[0, 1]$ into ten equal slices: it falls in slice 3, $[0.3, 0.4)$. Split that slice into ten: it falls in slice 7, $[0.37, 0.38)$. Split again: slice 2.

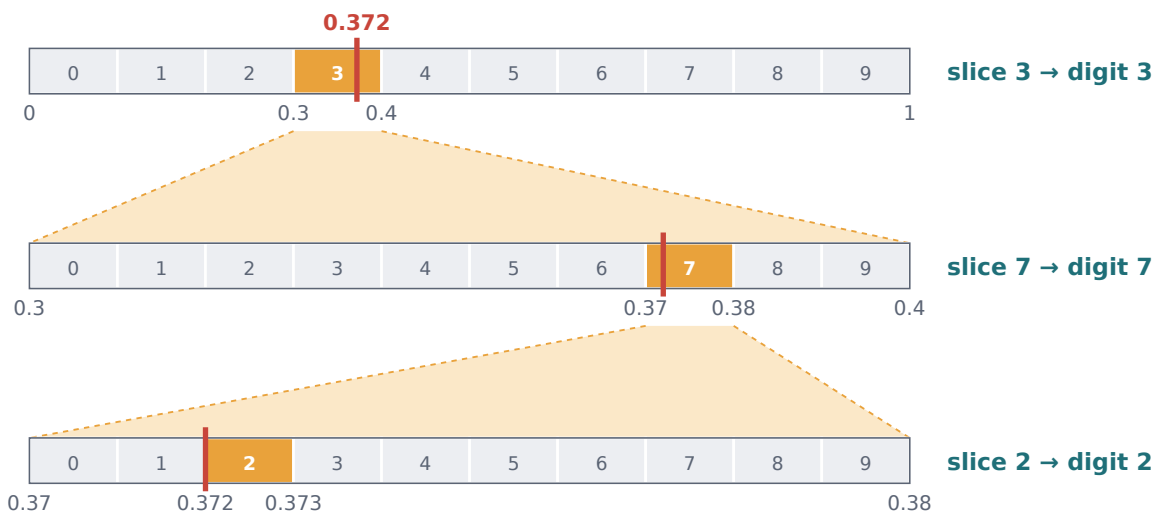


Figure 2. Zooming into the slice that contains 0.372. The slice numbers 3, 7, 2 are its decimal digits.

This is simply what positional notation means. The first digit after the point says which tenth the number is in, the second says which tenth of that tenth, and so on.

3. Binary digits are halving answers

Binary is the same scheme with two slices at each level, so every “which slice?” answer is 0 (lower half) or 1 (upper half). A binary fraction $0.b_1b_2b_3\dots$ stands for:

$$b_1 \cdot 1/2 + b_2 \cdot 1/4 + b_3 \cdot 1/8 + b_4 \cdot 1/16 + \dots$$

b_1 is 1 exactly when the number is at least $1/2$, which is the upper half of $[0, 1]$. Once that half is fixed, b_2 adds $1/4$, which is half the width of the surviving interval, so b_2 answers “upper half of that half?”. Every later digit adds half the width of whatever interval is left. The digits and the halving answers are the same thing.

Here is the halving search for 0.6875 on $[0, 1]$:

Step	Interval	Midpoint	Test	Bit	Place value
1	$[0, 1)$	0.5	$0.6875 \geq 0.5$	1	$1/2$
2	$[0.5, 1)$	0.75	$0.6875 < 0.75$	0	$1/4$
3	$[0.5, 0.75)$	0.625	$0.6875 \geq 0.625$	1	$1/8$
4	$[0.625, 0.75)$	0.6875	$0.6875 \geq 0.6875$	1	$1/16$

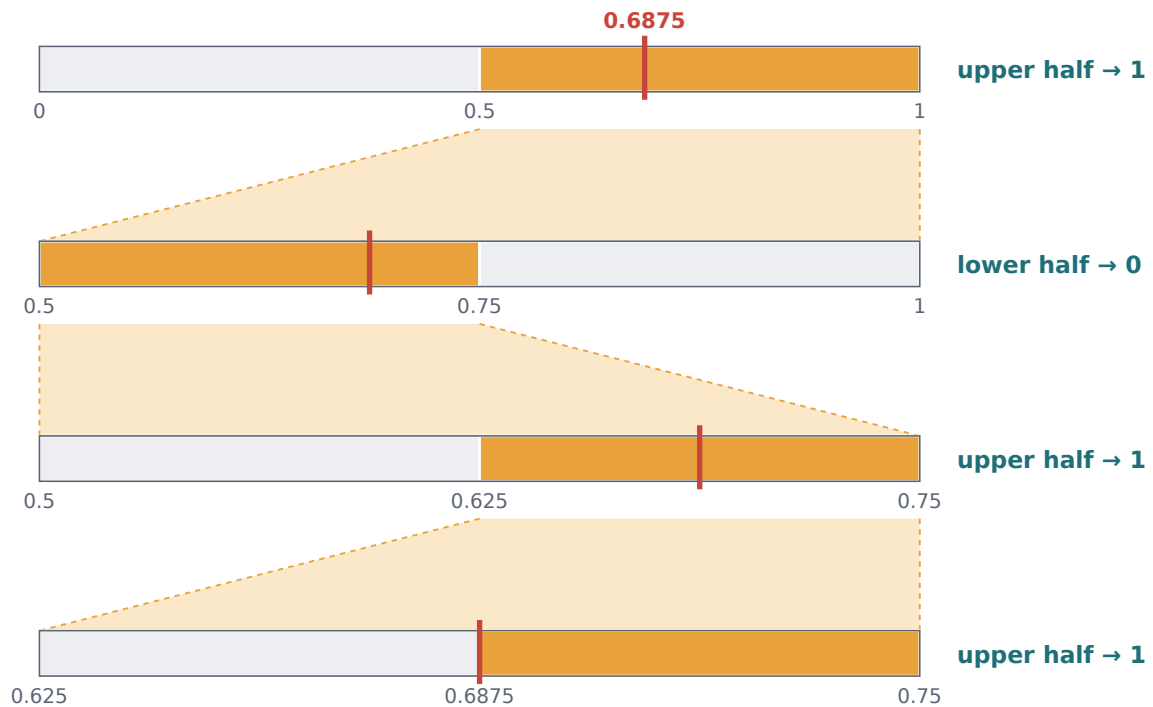


Figure 3. The same zoom with two slices per level. The answers read 1011.

Written in binary, $0.6875 = 1/2 + 1/8 + 1/16 = 0.1011$, which matches the four answers.

Ties are consistent. In step 4 the value sits exactly on the midpoint 0.6875 and goes to the upper half. In the binary expansion the corresponding digit is 1, so a “ $\geq$ goes up” rule and the digits agree even on boundaries.

4. Normalization: mapping any range onto [0, 1]

The digit correspondence works on $[0, 1]$, but coordinates live on other ranges. Normalization maps a range $[\min, \max]$ onto $[0, 1]$ with a linear formula:

$$\text{frac} = (x - \min) / (\max - \min)$$

It happens in two steps. Subtracting *min* shifts the range so it starts at 0. For longitude, $[-180, 180]$ becomes $[0, 360]$ and its width doesn't change. Dividing by the width, $\max - \min = 360$, then squeezes it to $[0, 1]$. The result says what fraction of the way from *min* to *max* the point is.

x (longitude)	x – min	frac = (x – min) / 360
–180	0	0
–90	90	0.25
0	180	0.5
90	270	0.75
100	280	0.7777...
180	360	1

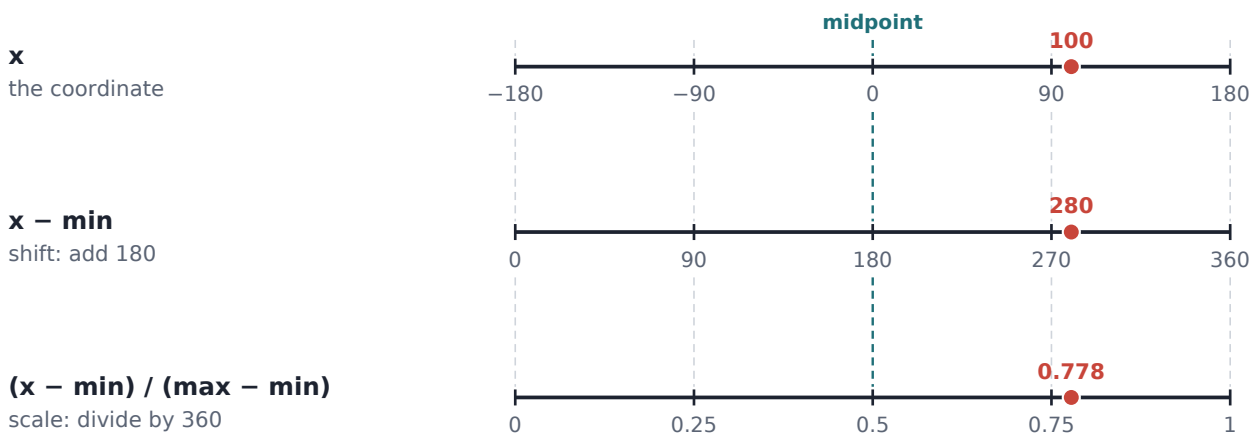


Figure 4. Shifting and scaling only relabel the line. Every tick keeps its relative position, so the midpoint stays the midpoint.

Because the map only shifts and stretches, it sends midpoints to midpoints. “Is $x \geq 0$?” becomes “is $\text{frac} \geq 0.5$?”, and “is $x \geq 90$?” becomes “is $\text{frac} \geq 0.75$?”. Every question in the halving search on the original range has a matching question on $[0, 1]$ with the same answer, so the bits are identical: they are the binary digits of frac .

Latitude works the same way on $[-90, 90]$, where $\text{max} - \text{min} = 180$.

5. Extracting the bits: multiply and truncate

A computer already stores frac in binary, so the halving answers are already in memory as the bits after the binary point. The only problem is getting them out as an integer.

Multiplying by 2 moves the binary point one place to the right, just as multiplying by 10 does in decimal. Multiplying by 2^n moves it n places, which brings the first n fractional bits into the integer part. Converting to an integer then truncates everything after the point, leaving exactly the first n halving answers.

$$k = \text{floor}(\text{frac} \times 2^n)$$

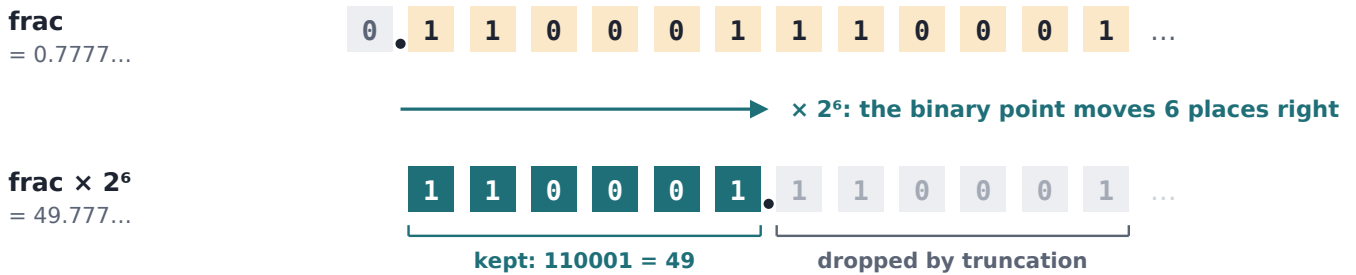


Figure 5. For longitude 100°, $\text{frac} = 0.110001110001\dots$ in binary. After multiplying by 2^6 , the six bits left of the point are the six halving answers.

For longitude 100°: $\text{frac} = 280 / 360 = 0.7777\dots$, which is $0.110001110001\dots$ in binary. Multiplying by $2^6 = 64$ gives $49.777\dots$, and truncating gives $49 = 110001$, the same six bits the loop produced in Figure 1.

Multiplying does not add accuracy. It only decides how many of the halving answers end up in the integer. The bits beyond the n -th still exist in frac , and truncation throws them away. Precision is chosen by n . (On a floating-point number the multiplication adjusts the exponent rather than literally shifting bits, but the effect is the same.)

Each extra bit halves the cell width. For longitude:

Bits (n)	Cells (2^n)	Cell width	Width at the equator
1	2	180°	~20,038 km
4	16	22.5°	~2,505 km
8	256	1.406°	~157 km
12	4,096	0.08789°	~9.8 km
16	65,536	0.005493°	~611 m
20	1,048,576	0.000343°	~38 m
26	67,108,864	0.00000536°	~0.60 m
32	4,294,967,296	0.0000000838°	~9 mm

6. The edge case at max

If x equals max , frac is exactly 1, and $1 \times 2^n = 2^n$. Cells are numbered 0 to $2^n - 1$, so this value falls one past the last cell and needs $n + 1$ bits. Clamp it to $2^n - 1$. The maximum edge then belongs to the last cell, and every other value is unaffected. Strictly speaking, normalization maps onto the closed interval $[0, 1]$, and this clamp is what handles the closing bracket.

7. Going back: from bits to a cell

Decoding reverses the steps. With n bits each cell has width $w = (\text{max} - \text{min}) / 2^n$, and cell k covers:

$$[\text{min} + k \cdot w , \text{min} + (k + 1) \cdot w)$$

The center, $\text{min} + (k + 0.5) \cdot w$, is the best single-point estimate, and it is never more than $w/2$ from the original value.

For $k = 49$ and $n = 6$ on longitude, $w = 360 / 64 = 5.625^\circ$. The cell is $[-180 + 49 \times 5.625, -180 + 50 \times 5.625) = [95.625^\circ, 101.25^\circ)$, the same interval Figure 1 ended with, and its center is 98.4375° .

8. Two axes: halving the world

Running the same procedure on longitude and latitude independently gives a column and a row. Together they pick out one rectangular cell of a grid laid over the whole map. With n bits per axis the grid has $2^n \times 2^n$ cells.

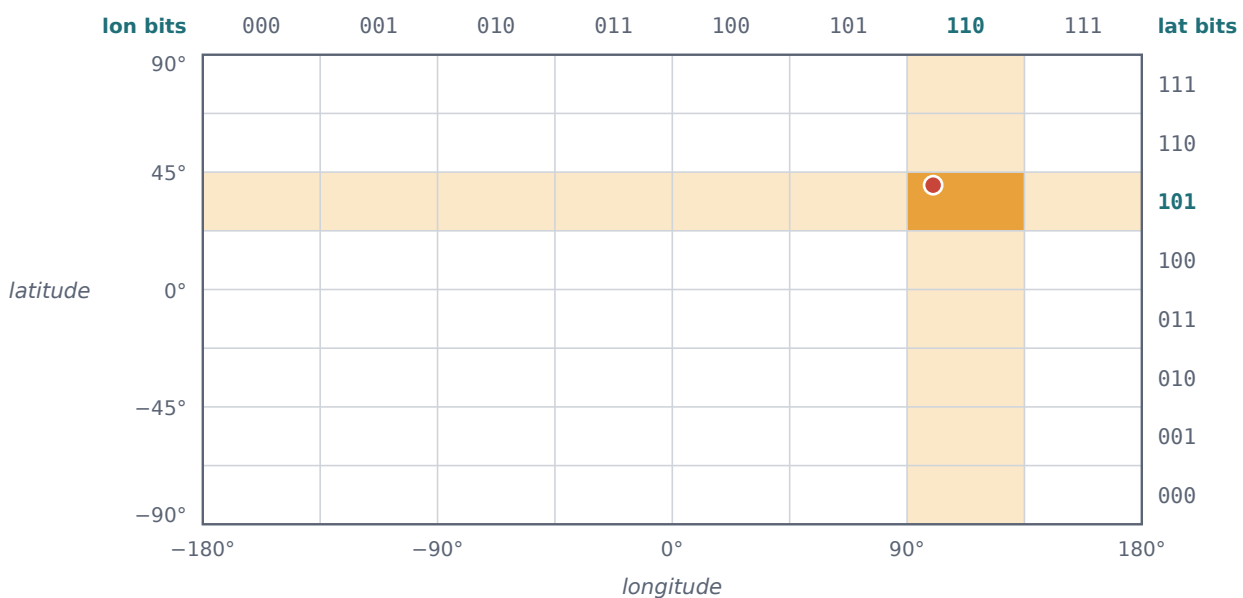


Figure 6. With 3 bits per axis the world is an 8×8 grid. Longitude 100° encodes to 110 (column 6) and latitude 40° to 101 (row 5). The point lies in their intersection.

Each extra bit per axis halves both dimensions of the cell, splitting every cell into four. How the two bit strings are then combined into a single key is a separate design choice. The per-axis encoding is exactly what the sections above describe.

9. The whole algorithm

In Go, with no loop over bits:

```
// encode returns the index of the cell containing x after [lo, hi]
// has been halved n times. The index has exactly n bits.
// Assumes lo <= x <= hi; validate input before calling.
func encode(x, lo, hi float64, n uint) uint64 {
    frac := (x - lo) / (hi - lo) // normalize to [0, 1]
    cells := uint64(1) << n      // 2^n cells
    k := uint64(frac * float64(cells)) // first n bits of frac, as an integer
    if k >= cells {                // x == hi lands one past the end
        k = cells - 1
    }
    return k
}

// decode returns the bounds of cell k.
func decode(k uint64, lo, hi float64, n uint) (cellLo, cellHi float64) {
    w := (hi - lo) / float64(uint64(1)<<n)
    cellLo = lo + float64(k)*w
    return cellLo, cellLo + w
}

// encode(100, -180, 180, 6) -> 49 (0b110001)
// decode(49, -180, 180, 6) -> 95.625, 101.25
```

In short

1. The answers to “which half?” are the binary digits of the point's position within its range.
2. Normalize: $\text{frac} = (x - \text{min}) / (\text{max} - \text{min})$.
3. Extract: $k = \text{floor}(\text{frac} \times 2^n)$, clamped to $2^n - 1$.
4. Decode: cell k spans $[\text{min} + k \cdot w, \text{min} + (k+1) \cdot w)$, with $w = (\text{max} - \text{min}) / 2^n$.
5. Precision is set by n alone.